Reviewer Pack — Minimal Rank of Hodge Integrals over Boolean Satisfiability …

Entry 4ae228116ef7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 14:22:12 UTC

Minimal Rank of Hodge Integrals over Boolean Satisfiability Solvable Instances

Entry ID: `4ae228116ef7` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 14:22:12 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Boolean Satisfiability

Statement:

> ['For each satisfiable instance of a Boolean satisfiability problem with at most 40 variables, the minimal rank of its associated Hodge integrals is bounded by the solution size of the instance.', 'Equivalently, for any given satisfiable instance I, there exists a constant c such that the minimal rank of the Hodge integral lattice L(I) satisfies min_rank(L(I)) ≤ c × |solution_size(I)|.', 'Moreover, if an instance I has a solution with size smaller than a threshold determined by its Hodge integral lattice, then I is not solvable.']

Rationale (proposer's reasoning):

> ['Hodge theory provides a framework for studying algebraic varieties and their topological properties. By associating Hodge integrals to Boolean satisfiability problems, we can potentially uncover hidden structures that influence the complexity of finding solutions.', 'If the minimal rank of the Hodge integral lattice is related to the solution size, it would suggest that certain topological features of the problem play a crucial role in determining its computational difficulty.', 'This connection could lead to new insights into the nature of NP-complete problems and potentially reveal polynomial-time algorithms for specific cases.']

Taxonomy category: `HODGE_ALGEBRAIC_GEOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `51c613554c117e37`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across 30 random seeds, the mean of min_rank(L(I)) divided by |solution_size(I)| for all satisfiable instances I with at most 40 variables is less than or equal to a constant c.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Hodge integrals' AND 'Boolean satisfiability' AND 'algebraic geometry' - 'solution size' IN BOOLEAN Satisfiability AND 'Hodge theory' AND 'minimal rank' - 'Boolean satisfiability problem' AND 'Hodge integral lattice' AND 'algebraic geometry'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n):
        variables = [f'x{i}' for i in range(n)]
        clauses = []
        for _ in range(2**n - 1):
            clause = random.sample(variables, random.randint(1, n))
            if random.choice([True, False]):
                clause = [f'not {v}' for v in clause]
            clauses.append(' or '.join(clause))
        return ' and '.join(clauses)

    def dpll(instance):
        # Simplified DPLL solver
        variables = set()
        for clause in instance.split(' and '):
            for literal in clause.split(' or '):
                if literal.startswith('not'):
                    variables.add(literal[4:])
                else:
                    variables.add(literal)

        assignment = {v: None for v in variables}

        def solve():
```

```
        unassigned = [v for v, val in assignment.items() if val is None]
        if not unassigned:
            return all(assignment[v] == (v.startswith('not') != (instance.split(' and ')[i].sp

        v = unassigned[0]
        assignment[v] = True
        if solve():
            return True
        assignment[v] = False
        if solve():
            return True
        assignment[v] = None
        return False

    if solve():
        return len([v for v, val in assignment.items() if val])
    else:
        return 0

def hodge_integral_lattice(instance):
    # Constructive mapping to Hodge integral lattice (simplified)
    n = instance.count(' or ')
    return n

results = []
for n in range(1, 41):
    instances_tested = 0
    total_min_rank = 0
    for _ in range(30):
        instance = generate_instance(n)
        solution_size = dpll(instance)
        if solution_size > 0:
            min_rank = hodge_integral_lattice(instance)
            results.append((min_rank, solution_size))
            instances_tested += 1

if not results:
    return {
        "metric_name": "min_rank_per_solution_size",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "no_satisfiable_instances_found"
    }

mean_min_rank = sum(r[0] for r in results) / len(results)
mean_solution_size = sum(r[1] for r in results) / len(results)
support_fraction = Fraction(mean_min_rank, mean_solution_size).limit_denominator()

return {
    "metric_name": "min_rank_per_solution_size",
    "metric_value": mean_min_rank,
    "instances_tested": instances_tested,
    "conjecture_holds": support_fraction <= 10,
    "counterexample": ""
}
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = Fraction(sum(1 for r in results if r["conjecture_holds"]), len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_faili
    else:
        print("RESULT: INCONCLUSIVE no_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1a8c2eac.py", line 107, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1a8c2eac.py", line 74, in run_trial
    solution_size = dpll(instance)
                    ^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1a8c2eac.py", line 58, in dpll
    if solve():
       ^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1a8c2eac.py", line 50, in solve
    if solve():
       ^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1a8c2eac.py", line 46, in solve
    return all(assignment[v] == (v.startswith('not') != (instance.split(' and ')[i].split(' or ')[j].s
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1a8c2eac.py", line 46, in <genexpr>
    return all(assignment[v] == (v.startswith('not') != (instance.split(' and ')[i].split(' or ')[j].s
                                 ^
NameError: cannot access free variable 'v' where it is not associated with a value in enclosing scope
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete its intended task of verifying the conjecture. | next: Re-run the test code to ensure it completes successfully and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11321
2	preregistration	ollama_remote	glm4:latest	0	0	5539
3	novelty	ollama_remote	glm4:latest	0	0	4778
4	novelty	ollama_remote	glm4:latest	0	0	5326
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15430
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6805
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8139
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24072
9	judge	ollama_remote	glm4:latest	0	0	47321

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 128730 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4ae228116ef7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4ae228116ef7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4ae228116ef7.tar.gz` (if generated)